Semesterprojekt – Bewertungshandreichung

Web-Applikationen · WS 2026/27 · Prof. Dr. Pascal Laube · HTWG Konstanz

---

Diese Handreichung erklärt, **wie das Semesterprojekt benotet wird**. Der Schein hängt an den vier Labortestaten und ist davon getrennt. Die vier Meilensteine (M1–M4) sind Pflichtabgaben und die Voraussetzung für die Note. Die Note selbst ergibt sich aus den sechs Bewertungsdimensionen auf dieser Seite, angewendet auf die fertige Anwendung und die schriftliche Ausarbeitung.

Lest die Handreichung am besten früh. Sie ist gleichzeitig eine Checkliste dafür, worauf es am Ende ankommt, und damit eine Hilfe für die Entscheidungen, die ihr schon im Semester trefft.

Überblick

Das Projekt wird entlang von **6 Dimensionen** bewertet, jede mit **0 bis 3 Punkten**. Damit sind insgesamt **18 Punkte** erreichbar.

1. **Systemverständnis und Architekturentscheidungen** (max. 3 Punkte)
2. **Qualität der Full-Stack-Umsetzung** (max. 3 Punkte)
3. **Domäneneignung und Eigenleistung** (max. 3 Punkte)
4. **Code- und Projektqualität** (max. 3 Punkte)
5. **Reflexionstiefe** (max. 3 Punkte)
6. **Kursbezug** (Konzepte aus den Vorlesungen) (max. 3 Punkte)

Gesamt: 18 Punkte

> Die Note setzt voraus, dass alle vier Meilensteine (M1–M4) abgegeben sind. Die Meilensteine fließen selbst nicht in die Note ein, sie sind die Eintrittskarte.

Woraus ich die Punkte ableite

Jede Dimension bewerte ich aus einer bestimmten Quelle. Manches sehe ich nur im laufenden System, manches nur in der Ausarbeitung, manches im Repository. Wenn eine Quelle fehlt, fehlen auch die Punkte, selbst wenn der Rest gut ist.

- **1 · Systemverständnis** — Ausarbeitung (Architektur, Begründungen), Code
- **2 · Full-Stack-Umsetzung** — Laufende App, Repository
- **3 · Domäneneignung** — App und Ausarbeitung (Projektidee)
- **4 · Code- und Projektqualität** — Repository (Code, Commits, README, Setup)
- **5 · Reflexionstiefe** — Ausarbeitung (Reflexionskapitel)
- **6 · Kursbezug** — Code und Ausarbeitung gemeinsam

Die 6 Dimensionen im Detail

1 · Systemverständnis und Architekturentscheidungen

Es geht darum, ob ihr die Anwendung *konzeptuell durchdacht* habt, oder ob Frontend, Backend und Datenbank einfach zusammengesteckt wurden, bis es lief.

0 Punkte

Keine oder rein beschreibende Dokumentation; Architekturentscheidungen fehlen vollständig.

1 Punkt

Architektur ist beschrieben, aber Entscheidungen nicht begründet; wirkt nach Schema.

2 Punkte

Aufteilung in Frontend, Backend und Datenhaltung ist erkennbar reflektiert; Stack- und Designentscheidungen sind begründet.

3 Punkte

Klare, begründete Architektur; Datenmodell und API-Schnitt sind durchdacht; Alternativen wurden erwogen (z. B. SPA statt SSR, eigene REST-API statt externer Drittanbieter-API); Abwägungen und Grenzen sind ehrlich benannt.

Worauf ich schaue: Erklärt eure Ausarbeitung, *warum* die App so geschnitten ist, wie sie ist? Passt das Datenbankschema zur Domäne? Ist der API-Schnitt zwischen Client und Server sinnvoll gewählt, nicht nur irgendwie da?

Beispiel:

- *Schwach:* „Wir haben React, Express und Prisma benutzt." Eine Aufzählung ohne Begründung.
- *Stark:* „Wir haben uns gegen SSR entschieden, weil die App eine SPA hinter dem Login ist und wir mit React Router und einem eigenen Express-Backend gebaut haben. Unsere Daten modellieren wir relational in Prisma: Nutzer, Buchungen und Räume mit festen Beziehungen im `schema.prisma`."

---

2 · Qualität der Full-Stack-Umsetzung

Eine App, die nur eine Liste rendert und sonst nichts persistiert, ist keine überzeugende Web-Applikation. Hier zählt, ob der Stack end-to-end wirklich trägt.

0 Punkte

Kein funktionierender Durchstich; Frontend ohne echtes Backend, oder Backend ohne nutzbares Frontend.

1 Punkt

Grundfunktion vorhanden, aber brüchig: Daten werden nicht zuverlässig persistiert, Auth ist Fassade, oder Fehlerfälle brechen die App.

2 Punkte

Durchgängiger Datenfluss Client → API → Datenbank und zurück; Auth schützt echte Routen, Rollen werden **im Backend** geprüft; Lade- und Fehlerzustände sind sichtbar behandelt.

3 Punkte

Runde Full-Stack-Umsetzung: mehrere zusammenspielende Features, saubere REST-Endpunkte, robustes Verhalten bei Fehlern und Grenzfällen; die App fühlt sich wie ein echtes Produkt an, nicht wie eine Demo.

Worauf ich schaue: Ich registriere mich, lege Daten an, lade neu, melde mich wieder an, provoziere einen Fehler. Hält die App das aus? Landen die Daten wirklich in der Datenbank, und sind geschützte Bereiche tatsächlich geschützt?

Beispiel:

- *Schwach:* Es gibt einen Login-Button, aber jede Route ist auch ohne Token erreichbar. Angelegte Einträge sind nach einem Reload wieder weg.
- *Stark:* Das Token wird serverseitig geprüft, geschützte Endpunkte antworten ohne gültige Session mit 401 und ohne passende Rolle mit 403. Angelegte Einträge überleben Reload und Server-Neustart, weil sie in der Datenbank liegen.

---

3 · Domäneneignung und Eigenleistung

Ein Semesterprojekt soll euer eigenes Werk sein, kein Tutorial mit neuem Namen.

0 Punkte

Tutorial-Klon (Todo-App, Standard-Shop) minimal angepasst; kein erkennbarer eigener Beitrag.

1 Punkt

Grundidee aus einem Tutorial, aber erkennbare Erweiterungen; Domäne generisch.

2 Punkte

Eigene Domäne, eigene Problemstellung; die App löst eine konkrete Aufgabe, die nicht 1:1 als Standard-Tutorial existiert.

3 Punkte

Durchdachte, originelle Domäne; ihr habt eigenständig ein sinnvolles Problem identifiziert, modelliert und in eine nutzbare App überführt.

Worauf ich schaue: Könnte jemand euer Projekt als fertiges Tutorial für genau dieses Problem online finden, oder habt ihr es selbst erfunden und gestaltet?

Beispiel:

- *Schwach:* Die klassische Todo-App mit anderem Titel und anderen Farben.
- *Stark:* Eine App für die Dienstplanung eines Kleintierheims, mit Schichten, Verfügbarkeiten der Helfenden und einer Konflikterkennung bei Doppelbelegung.

---

4 · Code- und Projektqualität

Lässt sich das Projekt reproduzieren? Ist der Code so geschrieben, dass jemand anderes daran weiterarbeiten könnte?

0 Punkte

App läuft nicht oder nur mit erheblichem undokumentiertem Aufwand.

1 Punkt

App läuft nach manuellem Eingriff; Code schwer lesbar; Commit-Historie dünn oder nur kurz vor den Abgaben.

2 Punkte

App startet reproduzierbar nach README; Code strukturiert; Tests vorhanden; Commits zeigen inkrementellen Fortschritt über das Semester.

3 Punkte

Professioneller Aufbau: klare Modul- und Komponentenstruktur, sprechende Commits, Setup ohne Handarbeit, aussagekräftige Tests dort, wo das Risiko liegt, keine Secrets im Repo (auch keine Fallback-Secrets im Code).

Worauf ich schaue: Ich clone das Repo, folge der README und starte die App. Wie weit komme ich ohne Rückfragen?

Beispiel:

- *Schwach:* `npm run dev` bricht ab, weil eine `.env` fehlt, die nirgends erwähnt wird. Fast alle Commits stammen aus der Nacht vor der Abgabe.
- *Stark:* Die README führt in wenigen Schritten durch das Setup, eine `.env.example` zeigt alle nötigen Variablen, ein Testuser ist genannt. Die Commits ziehen sich über das ganze Semester.

---

5 · Reflexionstiefe

Der Unterschied zwischen einer guten und einer sehr guten Note liegt oft hier: Habt ihr über euer Projekt nachgedacht, oder habt ihr es nur beschrieben?

0 Punkte

Keine oder rein deskriptive Reflexion.

1 Punkt

Reflexion vorhanden, aber allgemein gehalten; kein Bezug zur eigenen App.

2 Punkte

Ehrliche Auseinandersetzung mit Grenzen und Schwächen der eigenen App.

3 Punkte

Tiefe Reflexion: Was würde unter echter Last oder mit echten Nutzern schiefgehen? Welche Sicherheits- oder Datenschutzannahmen wurden getroffen? Was würdet ihr beim nächsten Mal anders bauen? Konkret und selbstkritisch.

Worauf ich schaue: Schreibt ihr über eure konkrete App, oder könnten die Sätze für jedes beliebige Web-Projekt stehen?

Beispiel:

- *Schwach:* „Das Projekt hat uns viel Spaß gemacht und wir haben viel gelernt."

- *Stark:* „Unsere `requireAuth`-Middleware prüft das JWT serverseitig; im Frontend halten wir das Token im AuthContext statt in `localStorage`. Trotzdem laden wir alle Einträge auf einmal per `GET /api/items` – ab vielen Datensätzen würde serverseitiges Filtern oder Paginierung nötig werden."

---

6 · Kursbezug (Web-Konzepte aus dem Kurs)

Das Projekt soll zeigen, dass ihr die Konzepte aus dem Kurs nicht nur benutzt, sondern *verstanden* habt: semantisches HTML und CSS, HTTP, TypeScript, React mit Komponenten, Hooks und Router, sauberes **API-Design** (Ressourcen, Methoden, Statuscodes), ein eigenes Backend mit Schichten und **Persistenz**, bewusster Umgang mit **Zustand** (Client vs. Server, Sessions), **Authentifizierung und Autorisierung**, **Web Security** (z. B. XSS, CSRF, CORS) sowie eine sinnvolle **Teststrategie**.

0 Punkte

Kein erkennbarer Bezug zu Kursinhalten; wirkt wie ohne Kurs entstanden.

1 Punkt

Technologien werden benutzt, aber Konzepte (Komponentenschnitt, State-Management, REST-Design, Auth) werden nicht aktiv gestaltet.

2 Punkte

Zentrale Kurskonzepte sind bewusst eingesetzt; mindestens eines geht erkennbar über das Minimal-Tutorial hinaus (z. B. durchdachtes API-Design, saubere Rollenprüfung im Backend, sinnvolle Teststrategie).

3 Punkte

Mehrere Kurskonzepte sind auf die eigene Domäne übertragen und greifen sauber ineinander; aus der Ausarbeitung wird erkennbar, dass ihr versteht, *warum* ihr es so gebaut habt, nicht nur, dass es funktioniert.

Worauf ich schaue: Wenn ich die Ausarbeitung und den Code lese, erkenne ich, dass ihr den Kursinhalt auf euer Problem angewendet habt, oder nur die Bibliotheken importiert?

Beispiel:

- *Schwach:* React ist im Einsatz, aber die ganze App steckt in einer 600-Zeilen-Komponente, Daten werden über fünf Ebenen per Prop-Drilling durchgereicht.
- *Stark:* Sinnvoller Komponentenschnitt, ein AuthContext für die Session, ein Express-REST-Server mit Prisma und klar benannten Ressourcen sowie gezielte Vitest-Tests für die Kernlogik (z. B. die Konflikterkennung).

Selbsteinschätzung vor der Abgabe

Geht diese Liste vor der finalen Abgabe (M4) einmal durch. Wenn ihr überall ehrlich „ja" sagen könnt, seid ihr im sehr guten Bereich.

- ☐ Ein Fremder kann unsere App nach der README starten oder aufrufen, ohne nachzufragen.
- ☐ Registrierung, Login und ein geschützter Bereich funktionieren wirklich, nicht nur optisch. Rollen werden im Backend geprüft.
- ☐ Angelegte Daten überleben einen Reload und einen Neustart des Servers.

- ☐ Es gibt sichtbare Lade- und Fehlerzustände, nicht nur den Happy Path.
- ☐ Unsere Ausarbeitung begründet die wichtigsten Architekturentscheidungen, statt sie nur aufzuzählen.
- ☐ Das Reflexionskapitel benennt konkrete Schwächen *unserer* App und was wir anders machen würden.
- ☐ Ich habe über das ganze Semester sichtbar committet, nicht nur vor den Abgaben.
- ☐ Es liegen keine echten Secrets (API-Keys, Passwörter) im Repository.
- ☐ Mindestens drei bis fünf aussagekräftige Tests laufen durch.
- ☐ Die App ist reproduzierbar nach README startbar, und das ist dokumentiert.
- ☐ Ein 3–5-minütiges Demo-Video zeigt die App in Nutzung.

Häufige Missverständnisse

„Wenn wir alle vier Meilensteine abgeben, bekommen wir eine 1,0." Nein. Die Meilensteine sind die Voraussetzung für die Note. Die Note hängt davon ab, wie durchdacht, robust und reflektiert eure Anwendung und Ausarbeitung sind, nicht davon, dass alle Häkchen gesetzt sind.

„Eine größere App mit mehr Features ergibt automatisch eine bessere Note." Nein. Eine fokussierte, gut gebaute App mit ehrlicher Reflexion schlägt ein aufgeblähtes Projekt ohne Verständnis. Tiefe schlägt Breite.

„Hauptsache, es läuft im Demo." Zu wenig. Ob Daten wirklich persistiert werden, ob geschützte Routen halten und ob Fehlerfälle sauber behandelt sind, fällt in Dimension 2 und 4 stärker ins Gewicht als ein hübsches, aber brüchiges Frontend.

„Die Ausarbeitung ist Formsache." Im Gegenteil. Systemverständnis (1), Reflexion (5) und Kursbezug (6) bewerte ich maßgeblich anhand der Ausarbeitung. Eine dünne Doku kostet bis zur Hälfte der Punkte, auch wenn die App gut ist.

„KI-Tools sind verboten." Nein. Ihr dürft KI-Tools nutzen. Verstehen müsst ihr trotzdem, was ihr abgebt. In den Übungen und an der Ausarbeitung merke ich schnell, ob ihr euer eigenes System durchdrungen habt.

Schein und Note

Schein

- Grundlage: alle vier **Labortestate** bestanden. Wer bei mindestens 80 % der Vorlesungen anwesend war, dem wird eines erlassen.
- Ergebnis: bestanden / nicht bestanden

Projektnote

- Voraussetzung: alle vier Meilensteine (M1–M4) abgegeben
- Grundlage: fertige Anwendung + schriftliche Ausarbeitung (PDF), bewertet über die 6 Dimensionen (18 Punkte)
- Ergebnis: benotete Prüfungsleistung

Das Projekt ist eine **Einzelarbeit**, die Note wird individuell vergeben.

Fragen zur Bewertung gerne jederzeit per Mail oder in der Übung ansprechen.